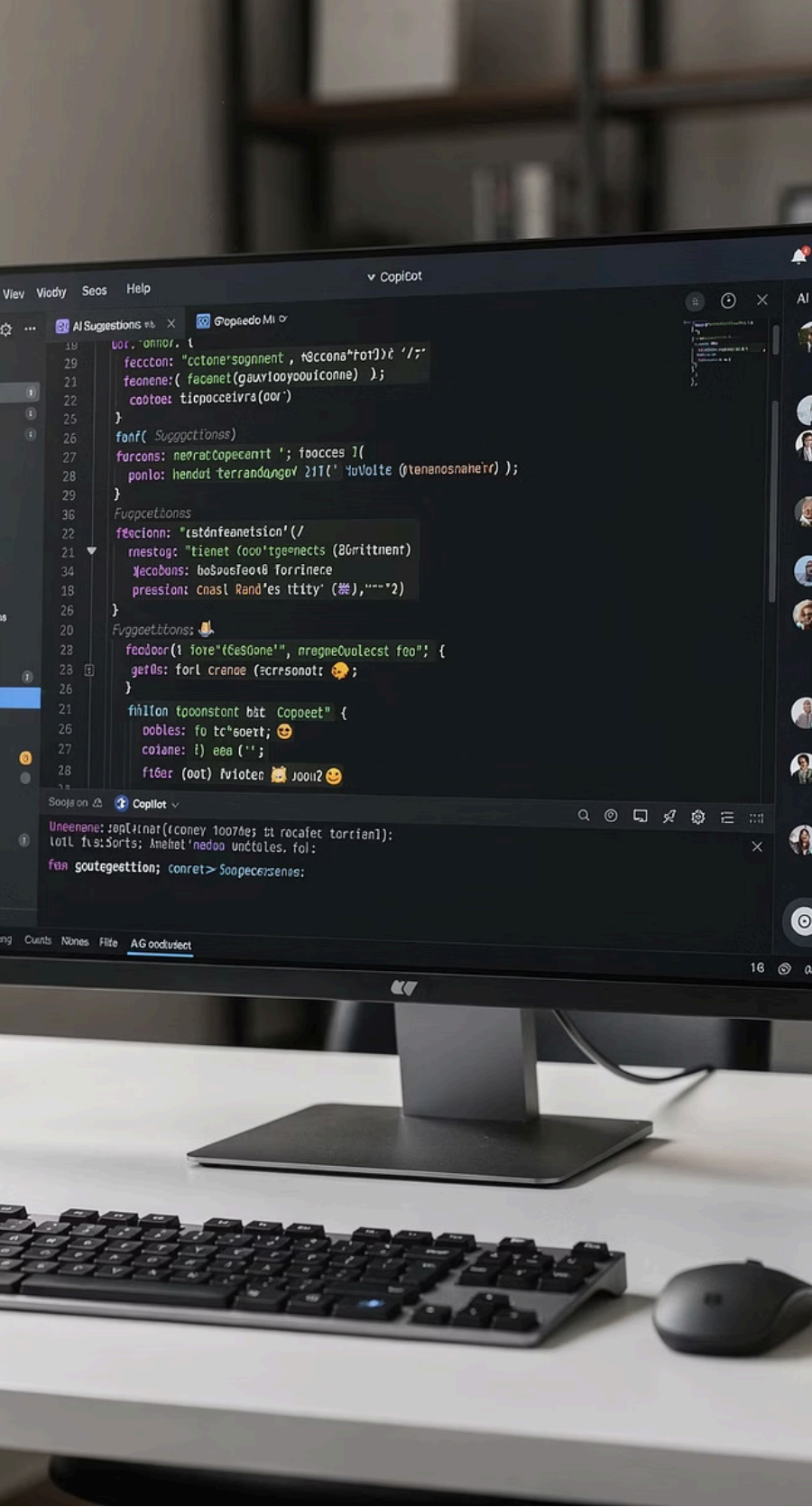




What if your AI Copilot had 25 specialized teammates?

Meet the harness that orchestrates them – a production-ready, open-source multi-agent system engineered for GitHub Copilot and Claude Code.



@orchestrator

The conductor – delegates every task, never writes code.



@planner

Breaks goals into focused subtasks for specialist agents.



@backend • @frontend • @migration

Parallel specialists – each with a narrow, clean context window.



@reviewer

Independent quality gate – nothing ships without passing review.

OPEN SOURCE

GITHUB COPILOT

CLAUDE CODE

[VIEW THE HARNESS →](#)



Single-agent AI hits a wall

When you force one AI agent to plan, write, test, review, and ship – it breaks down fast. Context bloats, quality drops, and nothing ships clean.

Context Collapse

Token bloat degrades reasoning quality over long sessions. The agent loses coherence before the task is done.

No Quality Gates

Without independent validation, broken logic ships. There's no checkpoint between generation and merge.

Self-Review Is Blind

An agent cannot reliably critique its own output. Confirmation bias is baked into single-agent loops.

Single Point of Failure

One bottleneck agent means one failure mode. Any degradation cascades across the entire workflow.

Multi-agent orchestration with mandatory quality chains

 Inspired by Anthropic's published orchestration patterns – smaller focused contexts, independent validation, better output quality at every stage.

01

Plan

@planner breaks the task into focused, routable subtasks.

02

Code

@backend, @frontend, @migration execute in parallel with clean contexts.

03

Test

@backend-tests and @e2e-tests validate every implementation independently.

04

Review

@reviewer is the final quality gate – nothing merges without approval.

Breaking the pipeline into discrete agent roles keeps each context small and purposeful. Every stage is independently verifiable, so errors are caught before they propagate – not after they ship.

35 Token-Optimized Skills

Skills are compressed, reusable knowledge modules loaded on demand. Agents pull only what they need – keeping context clean and inference costs low.

skills/

- |— react-vitest
- |— gherkin-format
- |— kubernetes-manifests
- |— ef-core
- |— security-review
- |— openapi-spec
- |— accessibility-audit

Compressed for Efficiency

Minimal tokens, maximum signal. No padding, no boilerplate.

Loaded on Demand

Agents fetch only the skills relevant to their active task.

Reusable Across Agents

One skill definition, shared by any agent that needs it.

✓ Knowledge is modular. Context stays clean.

Built on Anthropic's Harness Design Research

The orchestrator never writes code – it coordinates. Every agent has one job, one context, one responsibility. The harness enforces the chain.

 @orchestrator Top-level coordinator. Receives the goal, delegates to @planner, never touches implementation.	 @planner Decomposes the goal into discrete tasks. Routes each task to the right specialist agent.	 @backend • @frontend • @migration Three parallel specialists. Each runs in its own focused context – no cross-contamination.
 @backend-tests • @e2e-tests Independent test agents validate each implementation before anything moves forward.	 @reviewer Final quality gate. Runs a 7-dimension review: correctness, security, performance, style, coverage, docs, compliance.	

Why This Works

- Focused contexts**
Each agent sees only what it needs. No token bloat, no coherence loss.
- Independent validation**
No agent reviews its own work. Bias is structurally impossible.
- Mandatory quality chain**
Every path terminates at @reviewer. Nothing escapes the gate.

What It Replaces

- Single-agent overload**
One agent doing everything = context collapse + quality failure.
- Self-review loops**
Agents confirming their own output = blind spots at scale.
- Manual code review**
Human bottlenecks replaced by automated, consistent review agents.

25 Specialized Agents

Each agent owns one job. Focused context windows, clear responsibilities, no role bleed. From scaffolding to security audits – there's an agent for it.

@backend

.NET CRUD generation with repository patterns, service layers, and DTOs.

@frontend

React + MUI implementation with component structure and accessibility baked in.

@reviewer

7-dimension code review: correctness, security, performance, style, coverage, docs, and compliance.

@migration

EF Core migration generation, schema diffing, and rollback-safe scripts.

@orchestrator

Coordinates the full pipeline – delegates, monitors, and synthesizes results without writing code.

Plus: @tdd-red, @tdd-green, @tdd-refactor, @e2e, @planner, @docs, @accessibility, @devops, @security, @scaffold, and more.

7 Hooks That Protect Every Session

Hooks run automatically – before, during, and after every agent interaction. You don't configure them per session. They're always on.



secrets-scanner

Blocks credential and API key leakage before it reaches output.



governance-audit

Validates that every agent action aligns with defined governance rules.



dependency-license-checker

Flags non-compliant open-source licenses before they enter your codebase.



memory-snapshot

Captures session state for recovery and continuity across long workflows.



session-continuity

Restores context intelligently when a session is interrupted or resumed.



quality-verifier

Runs automated quality gates at key pipeline checkpoints.



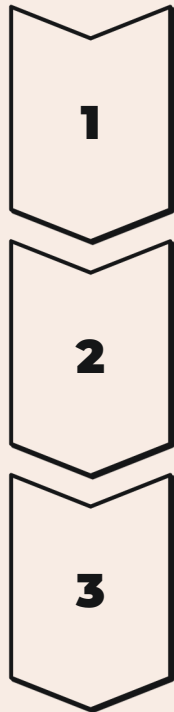
compliance-checker

Ensures output meets security and regulatory compliance standards automatically.

⚠️ Protection happens automatically – no manual intervention required.

Red → Green → Refactor (Automated)

Test-Driven Development isn't a discipline you enforce – it's a workflow baked into the agent pipeline. Three dedicated agents own each phase, cleanly separated.



● @tdd-red

Writes failing tests first. Defines expected behavior before a single line of implementation exists.

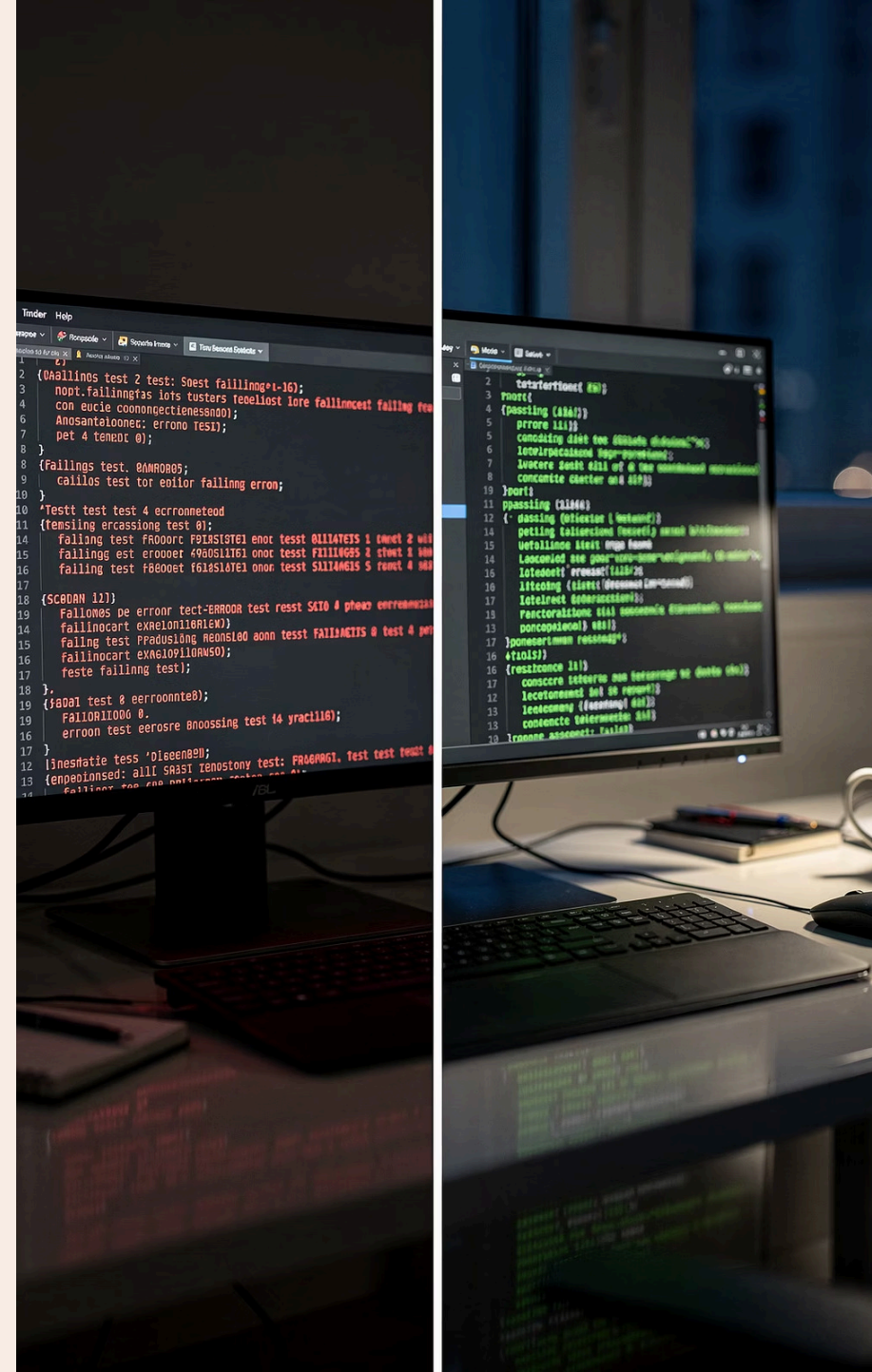
● @tdd-green

Creates the minimum implementation required to make failing tests pass. No over-engineering.

● @tdd-refactor

Improves design, removes duplication, and increases clarity – while keeping all tests green.

Built-in Test-Driven Development workflow – TDD is the default, not an afterthought.



One Repository. Infinite Possibilities.

Everything you need to run a production-grade multi-agent AI engineering workflow – versioned, composable, and ready to plug into your existing GitHub Copilot or Claude Code setup.

25

Specialized Agents

From backend to security, each agent owns exactly one responsibility.

35

Token-Optimized Skills

Modular, compressed knowledge loaded on demand to minimize context waste.

7

Automated Hooks

Always-on protection covering secrets, compliance, quality, and session state.

4

Plugins

Extend the system with composable integrations for your stack.

15

Prompt Templates

Battle-tested templates for common engineering workflows and code generation tasks.

[VIEW ON GITHUB](#)



Star the Repo

Ready to level up your AI Copilot?

Star it. Clone it. Symlink it. Ship better code – starting today.

```
git clone https://github.com/ALFRED-LUCIFER/awesome-skills-copilot
cd awesome-skills-copilot
# Symlink agents and skills into your .github/copilot directory
# Open in VS Code → GitHub Copilot picks up agents automatically
```



Star the Repo ★

Show support and help other engineers discover the project.



Symlink Into Your Workflow 🔗

Wire agents directly into your existing Copilot or Claude Code setup.

[VIEW ON GITHUB](#)

★ Star the Repo

Built for engineers who ship. Powered by the best AI tools available.

#GITHUBCOPILOT

#AIAGENTS

#CLAUDECODE

#DEVELOPERTOOLS

#OPENSOURCE



Clone & Explore 📦

Browse 25 agents, 35 skills, 7 hooks, and 4 plugins locally.



Build With It 🚀

Run your first orchestrated multi-agent session and never go back.